PROJECT DOCUMENTATION

Human Activity Recognition Using LSTM Neural Network

A Deep Learning Classification Project

Prepared by	Shafaat Ullah
Project domain	Artificial Intelligence / Deep Learning
Model	Long Short-Term Memory (LSTM)
Dataset	Human Activity Recognition with Smartphones
Platform	Google Colab and Python

Note: measured performance values are intentionally left blank until copied from the final executed notebook.

1. Project Overview

Human Activity Recognition (HAR) is the task of identifying a person's activity from sensor data. This project uses an LSTM neural network to classify smartphone-sensor feature data into activity categories such as walking, walking upstairs, walking downstairs, sitting, standing, and laying.

The workflow is implemented in Google Colab and covers dataset loading, data preparation, model training, evaluation, visualization, prediction, and saving model-related output files.

2. Objectives

- Load and inspect the HAR dataset.
- Prepare activity labels and numeric input features.
- Split data into training and validation subsets.
- Scale features and reshape the inputs for the LSTM model.
- Train a multi-class neural network and monitor validation performance.
- Evaluate results using accuracy, loss, classification metrics, and a confusion matrix.
- Save the trained model and export prediction results.

3. Dataset Description

The Human Activity Recognition with Smartphones dataset contains sensor measurements collected from smartphone accelerometers and gyroscopes while participants performed everyday activities. It is widely used as a benchmark for activity classification.

Dataset reference: Kaggle — Human Activity Recognition with Smartphones.

The exact files and label names should be confirmed from the dataset version loaded in the final notebook.

4. Tools and Technologies

- Python
- Google Colab / Jupyter Notebook
- pandas and NumPy for data manipulation
- scikit-learn for train/validation splitting, scaling, label encoding, and metrics
- TensorFlow / Keras for the LSTM model
- Matplotlib and Seaborn for plots
- joblib or pickle for saving preprocessing objects, depending on the notebook implementation.

5. Methodology

Data loading: Download or extract the dataset and load feature and label files into data frames.

Inspection and cleaning: Review data dimensions, column names, missing values, and target labels. Retain numeric features and align labels with the correct rows.

Label preparation: Encode activity names as class IDs and convert labels to one-hot vectors for multi-class training.

Data splitting: Use a stratified 80/20 training-validation split so that activity proportions are similar in both subsets.

Scaling and reshaping: Fit a StandardScaler on training data, transform validation data, and reshape inputs into (samples, features, 1) for the LSTM implementation described.

Training: Train the model with the Adam optimizer and categorical cross-entropy loss. Early stopping and learning-rate reduction callbacks can stop training when validation progress stalls.

Evaluation: Review accuracy and loss, a classification report, a confusion matrix, and actual-versus-predicted examples.

6. LSTM Model Architecture

The model described in the notebook uses the following sequential layers:

Layer	Purpose
Input	Accepts reshaped sensor-feature input.
LSTM — 64 units, return_sequences=True	Learns patterns across positions in the input sequence and returns a sequence for the next recurrent layer.
Batch Normalization	Helps stabilize intermediate activations.
Dropout — 0.30	Regularization to reduce overfitting.
LSTM — 32 units	Learns a compact representation of the input.
Dropout — 0.30	Adds regularization during training.
Dense — 64 units, ReLU	Learns non-linear combinations of extracted features.
Dropout — 0.20	Adds further regularization.
Output Dense — class count, Softmax	Produces class probabilities for the activity categories.

7. Training Configuration

The notebook version discussed uses up to 30 epochs, a batch size of 64, the Adam optimizer, categorical cross-entropy loss, EarlyStopping, and ReduceLROnPlateau. The model may stop before the maximum epoch count if validation performance no longer improves.

Training settings can change if the notebook is edited. Use the final executed notebook as the authoritative source for exact settings.

8. Evaluation and Results

Evaluation includes validation accuracy and loss, precision, recall, F1-score, a confusion matrix, training curves, and example predictions.

Results note: The final measured accuracy and loss were not available when this report was prepared. Fill in the values from the executed Colab output rather than reporting an assumed result.

Metric	Final measured value
Validation accuracy	_____
Validation loss	_____
Best validation accuracy (if recorded)	_____
Epoch at early stopping	_____

Recommended figures: training/validation accuracy, training/validation loss, and the confusion matrix from the notebook.

9. Output Files

The notebook's export cells are intended to create these files if they complete successfully:
• **har_lstm_model.keras** — trained Keras model.
• **har_scaler.pkl** — fitted feature scaler.
• **har_label_encoder.pkl** — activity label encoder.
• **har_lstm_predictions.csv** — exported prediction results.

Verify the exact filenames in the Colab file browser before submitting the project.

10. Limitations

• Reshaping a feature vector into (features, 1) lets the model process feature positions as a sequence, but this is not equivalent to training on raw, time-windowed sensor readings.

• Performance depends on correct data loading, target identification, preprocessing, and label alignment.

• Validation performance may differ from performance on new users, devices, or environments.

• Per-class metrics and the confusion matrix should be reviewed to find activities that the model confuses.

11. Future Improvements

• Train on raw, windowed accelerometer and gyroscope readings to represent true temporal sequences.
• Compare LSTM with SimpleRNN, GRU, 1D-CNN, and CNN-LSTM models.
• Tune sequence length, hidden units, dropout, batch size, and learning rate.
• Evaluate on held-out users when possible.
• Export final metrics and plots automatically into the project report.

12. Conclusion

This project demonstrates a complete deep-learning workflow for smartphone-based human activity classification using an LSTM neural network. It includes dataset preparation, model training, evaluation, visualization, predictions, and saving model artifacts. The final report should be completed with the actual values and figures from the final Colab run.

References

1. Kaggle: Human Activity Recognition with Smartphones
2. TensorFlow / Keras Guide
3. scikit-learn Documentation